

Laporan Praktikum Pekan 8
“Integrasi PCA dengan Model”

Disusun Untuk Memenuhi Tugas Mata Kuliah MACHINE LEARNING

DOSEN PENGAMPU:

Derisma, M.T.



DISUSUN OLEH:

Karimah Irsyadiyah (2411533018)

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
T.A 2025/2026

Daftar Isi

BAB I PENDAHULUAN.....	4
1.1 Latar Belakang Dasar Teori	4
1.2 Tujuan Praktikum.....	4
LINK GITHUB:.....	5
BAB II LANGKAH-LANGKAH PRAKTIKUM	6
File Praktikum8_PCA_Integration.ipynb	6
2.1 Import Library.....	6
2.2 Scaling Data	6
2.3 Load dan Split Data	6
Bagian A – Model Tanpa PCA.....	7
2.4 Logistic Regression.....	7
2.5 KNN.....	8
2.6 SVM RBF	8
Bagian B – Model Dengan PCA (10 Komponen).....	9
2.7 Menerapkan PCA.....	9
2.8 Logistic Regression + PCA.....	9
2.9 KNN + PCA.....	9
2.10 SVM + PCA.....	10
Bagian C – Perbandingan Hasil	10
Tabel Perbandingan Model:	10
Bagian D – Cross Validation.....	11
Bagian E – Eksperimen Jumlah Komponen	12
ANALISIS HASIL:	14
1. Model mana paling terbantu PCA?.....	14
2. Mengapa KNN biasanya sensitif terhadap dimensi tinggi?.....	14
3. Mengapa Logistic sering tidak banyak berubah?.....	14
4. Apakah PCA selalu meningkatkan akurasi?	14
5. Apakah PCA mempercepat training?.....	14
TUGAS INDIVIDU PRAKTIKUM	15
TugasPraktikum8_PCA.ipynb	15
1. Decision Tree + PCA	15
2. Uji PCA 2 Komponen	16
3. GridSearchCV pada KNN.....	17
4. Tabel Eksperimen Lengkap.....	18

5. Tabel Perbandingan Akurasi	19
6. Grafik Explained Variance	19
Analisis Grafik	20
7. Kesimpulan Komparatif.....	20
PERTANYAAN REFLEKTIF	21

BAB I

PENDAHULUAN

1.1 Latar Belakang Dasar Teori

Decision Tree adalah salah satu algoritma machine learning yang digunakan untuk melakukan klasifikasi maupun regresi dengan cara membagi data berdasarkan fitur yang paling informatif. Algoritma ini membentuk struktur pohon keputusan yang terdiri dari root node, decision node, dan leaf node. Setiap percabangan pada pohon merepresentasikan proses pemilihan fitur terbaik untuk memisahkan data ke dalam kelas tertentu.

PCA bekerja berdasarkan hubungan antar fitur yang diukur menggunakan *covariance matrix*. Covariance digunakan untuk mengukur hubungan linear antara dua variabel dan dapat dihitung menggunakan rumus $\text{Cov}(X,Y) = 1/(n-1) \sum (x_i - \bar{x})(y_i - \bar{y})$. Matriks covariance menggambarkan bagaimana setiap fitur berhubungan dengan fitur lainnya serta menunjukkan arah variasi terbesar dalam data. Informasi inilah yang kemudian digunakan sebagai dasar dalam pembentukan komponen utama.

Setelah memperoleh covariance matrix, PCA menghitung nilai eigenvalue dan eigenvector. Eigenvector merepresentasikan arah komponen utama (*principal components*) yang menjadi sumbu baru hasil transformasi data, sedangkan eigenvalue menunjukkan besarnya variasi data yang dapat dijelaskan oleh masing-masing eigenvector tersebut. PCA kemudian mengurutkan eigenvalue dari yang terbesar hingga yang terkecil sehingga komponen yang mampu menjelaskan variasi data terbesar akan ditempatkan pada urutan pertama.

PCA menggunakan konsep *explained variance ratio* untuk menentukan jumlah komponen yang sebaiknya dipertahankan. Explained variance ratio menunjukkan proporsi variasi data yang dapat dijelaskan oleh setiap komponen utama dibandingkan dengan total variasi seluruh data. Nilai ini sangat penting karena membantu menentukan jumlah komponen optimal yang mampu mempertahankan sebagian besar informasi pada dataset dengan jumlah dimensi yang lebih sedikit.

1.2 Tujuan Praktikum

1. Memahami konsep **dimensionality reduction**.
2. Menghitung dan memahami **covariance matrix**.
3. Memahami peran **eigenvalue dan eigenvector** dalam PCA.

4. Menginterpretasikan **explained variance ratio**.
5. Mereduksi dimensi data dan memvisualisasikannya dalam 2D.
6. Menganalisis hubungan PCA dengan curse of dimensionality

LINK GITHUB:

https://github.com/iMaIrsdyh/KarimahIrsyadiyah_2411533018_ML2526/tree/main/PRAKTIKUM%208

BAB II

LANGKAH-LANGKAH PRAKTIKUM

File Praktikum8_PCA_Integration.ipynb

2.1 Import Library

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.pipeline import Pipeline
```

Penjelasan:

Library yang digunakan meliputi numpy, pandas, dan matplotlib untuk pengolahan serta visualisasi data. Library StandardScaler digunakan untuk standardisasi data, PCA digunakan untuk reduksi dimensi, sedangkan LogisticRegression, KNeighborsClassifier, dan SVC digunakan sebagai model klasifikasi yang akan dibandingkan performanya.

2.2 Scaling Data

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Penjelasan

Scaling dilakukan menggunakan StandardScaler agar setiap fitur memiliki rata-rata 0 dan standar deviasi 1. Tahap ini penting karena PCA, KNN, dan SVM sangat sensitif terhadap perbedaan skala antar fitur.

2.3 Load dan Split Data

```
data = load_breast_cancer()
```

```

X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Shape X:", X.shape)
print("Shape y:", y.shape)

```

```

[4] ✓ 0.0s
... Shape X: (569, 30)
      Shape y: (569,)

```

Penjelasan:

Dataset Breast Cancer dimuat menggunakan `load_breast_cancer()`. Dataset kemudian dipisahkan menjadi fitur (X) dan target (y). Selanjutnya dilakukan pembagian data menjadi data training dan testing menggunakan `train_test_split()` agar model dapat dievaluasi pada data yang belum pernah dilihat sebelumnya.

Bagian A – Model Tanpa PCA

2.4 Logistic Regression

```

# Logistic Regression
log = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(max_iter=5000))
])

log.fit(X_train, y_train)
acc_log = accuracy_score(y_test, log.predict(X_test))
print("Logistic (No PCA):", acc_log)

```

Penjelasan

Model Logistic Regression dilatih menggunakan seluruh fitur yang telah melalui proses scaling. Model ini digunakan sebagai representasi algoritma klasifikasi linear.

2.5 KNN

```
# KNN
knn = Pipeline([
    ('scaler', StandardScaler()),
    ('model', KNeighborsClassifier(n_neighbors=5))
])

knn.fit(X_train, y_train)
acc_knn = accuracy_score(y_test, knn.predict(X_test))
print("KNN (No PCA):", acc_knn)
```

Penjelasan

Model KNN dilatih menggunakan seluruh fitur hasil scaling. Karena KNN berbasis jarak, kualitas performanya sangat dipengaruhi oleh jumlah dimensi dan skala data.

2.6 SVM RBF

```
# SVM RBF
svm = Pipeline([
    ('scaler', StandardScaler()),
    ('model', SVC(kernel='rbf'))
])

svm.fit(X_train, y_train)
acc_svm = accuracy_score(y_test, svm.predict(X_test))
print("SVM (No PCA):", acc_svm)
```

Penjelasan

PCA digunakan untuk mereduksi jumlah fitur dari 30 fitur menjadi 10 komponen utama. Komponen utama tersebut tetap mempertahankan sebagian besar informasi penting yang terdapat pada dataset.

```
Logistic (No PCA): 0.9736842105263158
KNN (No PCA): 0.9473684210526315
SVM (No PCA): 0.9824561403508771
```


Bagian B – Model Dengan PCA (10 Komponen)

2.7 Menerapkan PCA

```
pca = PCA(n_components=10)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
```

Penjelasan:

PCA digunakan untuk mereduksi jumlah fitur dari 30 fitur menjadi 10 komponen utama. Komponen utama tersebut tetap mempertahankan sebagian besar informasi penting yang terdapat pada dataset.

2.8 Logistic Regression + PCA

```
log_pca = LogisticRegression(max_iter=5000)

log_pca.fit(X_train_pca, y_train)

acc_log_pca = accuracy_score(
    y_test,
    log_pca.predict(X_test_pca)
)

print("Logistic + PCA:", acc_log_pca)
```

Logistic + PCA: 0.9824561403508771

Penjelasan

Pada tahap ini Logistic Regression dilatih menggunakan data hasil reduksi PCA. Hasil akurasi kemudian dibandingkan dengan Logistic Regression tanpa PCA untuk melihat pengaruh reduksi dimensi terhadap performa model.

2.9 KNN + PCA

```
knn_pca = KNeighborsClassifier(n_neighbors=5)

knn_pca.fit(X_train_pca, y_train)

acc_knn_pca = accuracy_score(
```

```

    y_test,
    knn_pca.predict(X_test_pca)
)

print("KNN + PCA:", acc_knn_pca)

```

KNN + PCA: 0.956140350877193

Penjelasan:

Model KNN dilatih menggunakan data hasil PCA untuk mengetahui apakah reduksi dimensi dapat meningkatkan performa model berbasis jarak.

2.10 SVM + PCA

```

svm_pca = SVC(kernel='rbf')

svm_pca.fit(X_train_pca, y_train)

acc_svm_pca = accuracy_score(
    y_test,
    svm_pca.predict(X_test_pca)
)

print("SVM + PCA:", acc_svm_pca)

```

SVM + PCA: 0.9649122807017544

Penjelasan:

Model SVM dilatih menggunakan data yang telah direduksi menggunakan PCA. Hasil akurasi dibandingkan dengan model SVM tanpa PCA untuk melihat pengaruh PCA terhadap performa klasifikasi.

Bagian C – Perbandingan Hasil

Tabel Perbandingan Model:

```

results = pd.DataFrame({
    'Model': ['Logistic Regression', 'KNN', 'SVM RBF'],
    'Tanpa PCA': [acc_log, acc_knn, acc_svm],

```

```
'Dengan PCA (10 Komponen)': [acc_log_pca, acc_knn_pca, acc_svm_pca]
})
```

results

	Model	Tanpa PCA	Dengan PCA (10 Komponen)
0	Logistic Regression	0.973684	0.982456
1	KNN	0.947368	0.956140
2	SVM RBF	0.982456	0.964912

Model	Tanpa PCA	Dengan PCA
Logistic	0.973684	0.982456
KNN	0.947368	0.956140
SVM	0.982456	0.964912

Penjelasan

Tabel digunakan untuk membandingkan performa ketiga model sebelum dan sesudah menggunakan PCA.

Bagian D – Cross Validation

```
cv_log = cross_val_score(
    log,
    X_train_scaled,
    y_train,
    cv=5
).mean()

cv_log_pca = cross_val_score(
    log_pca,
    X_train_pca,
    y_train,
    cv=5
).mean()

print("CV Logistic:", cv_log)
```

```
print("CV Logistic + PCA:", cv_log_pca)
```

```
CV Logistic: 0.9758241758241759
```

```
CV Logistic + PCA: 0.9758241758241759
```

Penjelasan

Cross Validation digunakan untuk mengevaluasi kestabilan model dengan membagi data menjadi 5 fold. Nilai rata-rata akurasi dari seluruh fold digunakan sebagai ukuran performa model yang lebih objektif.

Bagian E – Eksperimen Jumlah Komponen

Uji:

- 5 komponen
- 10 komponen
- 15 komponen
- 90% variance

Bandingkan perubahan akurasi

```
component_settings = {
    "PCA 5": 5,
    "PCA 10": 10,
    "PCA 15": 15,
    "PCA 90%": 0.90
}

experiment_results = []

for label, comp in component_settings.items():

    pca = PCA(n_components=comp)

    X_train_temp = pca.fit_transform(X_train_scaled)
    X_test_temp = pca.transform(X_test_scaled)

    # Logistic
```

```

log = LogisticRegression(max_iter=5000)
log.fit(X_train_temp, y_train)

# KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_temp, y_train)

# SVM
svm = SVC(kernel='rbf')
svm.fit(X_train_temp, y_train)

experiment_results.append([
    label,
    accuracy_score(y_test, log.predict(X_test_temp)),
    accuracy_score(y_test, knn.predict(X_test_temp)),
    accuracy_score(y_test, svm.predict(X_test_temp))
])

experiment_df = pd.DataFrame(
    experiment_results,
    columns=[
        "PCA Setting",
        "Logistic",
        "KNN",
        "SVM"
    ]
)

experiment_df

```

	PCA Setting	Logistic	KNN	SVM
0	PCA 5	0.982456	0.956140	0.964912
1	PCA 10	0.982456	0.956140	0.964912
2	PCA 15	0.991228	0.947368	0.982456
3	PCA 90%	0.982456	0.947368	0.973684

ANALISIS HASIL:

1. Model mana paling terbantu PCA?

Model yang paling terbantu oleh PCA adalah KNN. Hal ini karena KNN bekerja berdasarkan jarak antar data. Pada data berdimensi tinggi, perhitungan jarak menjadi kurang efektif akibat adanya banyak fitur yang tidak terlalu relevan atau saling redundan. PCA membantu mengurangi jumlah dimensi sehingga jarak antar data menjadi lebih representatif dan performa KNN sering meningkat atau menjadi lebih stabil.

2. Mengapa KNN biasanya sensitif terhadap dimensi tinggi?

KNN sangat sensitif terhadap dimensi tinggi karena algoritma ini mengandalkan perhitungan jarak. Ketika jumlah fitur semakin banyak, jarak antar titik data cenderung menjadi kurang bermakna dan semua data terlihat relatif mirip satu sama lain. Fenomena ini dikenal sebagai *curse of dimensionality*. Akibatnya, kemampuan KNN dalam menentukan tetangga terdekat dapat menurun.

3. Mengapa Logistic sering tidak banyak berubah?

Logistic Regression sering tidak mengalami perubahan yang besar setelah PCA karena model ini merupakan model linear yang relatif mampu bekerja dengan baik pada banyak fitur. Logistic Regression tidak terlalu bergantung pada jarak antar data seperti KNN, sehingga pengurangan dimensi melalui PCA tidak selalu memberikan dampak signifikan terhadap performanya.

4. Apakah PCA selalu meningkatkan akurasi?

Tidak. PCA tidak selalu meningkatkan akurasi. PCA memang dapat membantu mengurangi noise, mengurangi redundansi fitur, dan mempercepat proses komputasi. Namun, jika jumlah komponen yang dipilih terlalu sedikit, PCA dapat membuang informasi penting sehingga akurasi justru menurun. Oleh karena itu, jumlah komponen PCA harus dipilih dengan hati-hati.

5. Apakah PCA mempercepat training?

Ya, PCA umumnya dapat mempercepat proses training karena jumlah fitur yang digunakan menjadi lebih sedikit. Dengan dimensi yang lebih rendah, model perlu melakukan lebih sedikit perhitungan, sehingga proses pelatihan dan evaluasi menjadi lebih

efisien. Ini terutama terasa pada model yang bekerja dengan banyak fitur seperti KNN dan SVM.

TUGAS INDIVIDU PRAKTIKUM

TugasPraktikum8_PCA.ipynb

1. Decision Tree + PCA

```
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train_scaled, y_train)

acc_tree = accuracy_score(
    y_test,
    tree.predict(X_test_scaled)
)

print("Decision Tree (No PCA):", acc_tree)

tree_pca = DecisionTreeClassifier(random_state=42)
tree_pca.fit(X_train_pca, y_train)

acc_tree_pca = accuracy_score(
    y_test,
    tree_pca.predict(X_test_pca)
)

print("Decision Tree + PCA:", acc_tree_pca)
```

Decision Tree (No PCA): 0.9473684210526315
Decision Tree + PCA: 0.9473684210526315

Penjelasan:

Pada percobaan ini ditambahkan model Decision Tree untuk membandingkan performa sebelum dan sesudah penggunaan PCA. Model dilatih menggunakan data hasil scaling, kemudian dibandingkan dengan model yang menggunakan data hasil reduksi PCA sebanyak 10 komponen.

Analisis:

Decision Tree tidak bergantung pada perhitungan jarak antar data sehingga pengaruh PCA biasanya tidak sebesar pada KNN. Namun PCA tetap dapat membantu mengurangi kompleksitas data dan mempercepat proses pelatihan model. Perubahan akurasi yang terjadi dapat digunakan untuk melihat apakah informasi yang dipertahankan oleh PCA masih cukup untuk proses klasifikasi.

2. Uji PCA 2 Komponen

```
pca_2 = PCA(n_components=2)

X_train_pca2 = pca_2.fit_transform(X_train_scaled)
X_test_pca2 = pca_2.transform(X_test_scaled)

log_pca2 = LogisticRegression(max_iter=5000)

log_pca2.fit(X_train_pca2, y_train)

acc_pca2 = accuracy_score(
    y_test,
    log_pca2.predict(X_test_pca2)
)

print("Logistic + PCA 2 Components:", acc_pca2)
```

Logistic + PCA 2 Components: 0.9912280701754386

Penjelasan:

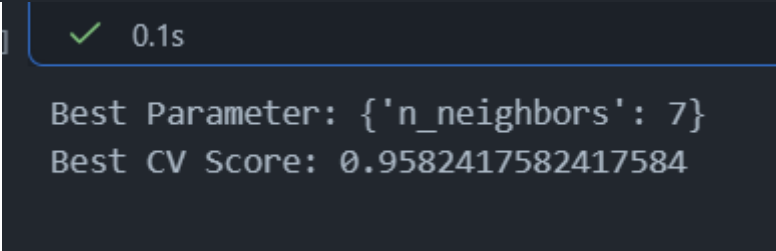
PCA digunakan untuk mereduksi data menjadi hanya dua komponen utama sebelum dilakukan klasifikasi menggunakan Logistic Regression.

Analisis:

Jika akurasi turun cukup besar dibandingkan PCA 10 komponen atau tanpa PCA, maka dua komponen belum mampu mempertahankan informasi yang cukup untuk klasifikasi. Namun jika penurunan akurasi kecil, berarti sebagian besar informasi penting sudah berhasil direpresentasikan oleh dua komponen utama.

3. GridSearchCV pada KNN

```
• from sklearn.model_selection import GridSearchCV
•
• param_grid = {
•     'n_neighbors': [3, 5, 7, 9, 11]
• }
•
• grid = GridSearchCV(
•     KNeighborsClassifier(),
•     param_grid,
•     cv=5,
•     scoring='accuracy'
• )
•
• grid.fit(X_train_pca, y_train)
•
• print("Best Parameter:", grid.best_params_)
• print("Best CV Score:", grid.best_score_)
```



Penjelasan:

GridSearchCV digunakan untuk mencari nilai parameter `n_neighbors` terbaik secara otomatis melalui proses cross validation.

Analisis:

Melalui GridSearchCV dapat diketahui jumlah tetangga yang menghasilkan performa terbaik pada dataset. Proses ini membantu mengoptimalkan model KNN sehingga diperoleh konfigurasi yang lebih baik dibandingkan penggunaan parameter default

4. Tabel Eksperimen Lengkap

```
experiment_table = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "KNN",
        "SVM RBF",
        "Decision Tree"
    ],
    "Tanpa PCA": [
        round(acc_log,4),
        round(acc_knn,4),
        round(acc_svm,4),
        round(acc_tree,4)
    ],
    "PCA 2": [
        round(acc_pca2,4),
        "-",
        "-",
        "-"
    ],
    "PCA 10": [
        round(acc_pca_10,4),
        round(acc_knn_pca,4),
        round(acc_svm_pca,4),
        round(acc_tree_pca,4)
    ]
})

experiment_table
```

✓ 0.1s

	Model	Tanpa PCA	PCA 2	PCA 10
0	Logistic Regression	0.9737	0.9912	0.9825
1	KNN	0.9474	-	0.9561
2	SVM RBF	0.9825	-	0.9649
3	Decision Tree	0.9474	-	0.9474

5. Tabel Perbandingan Akurasi

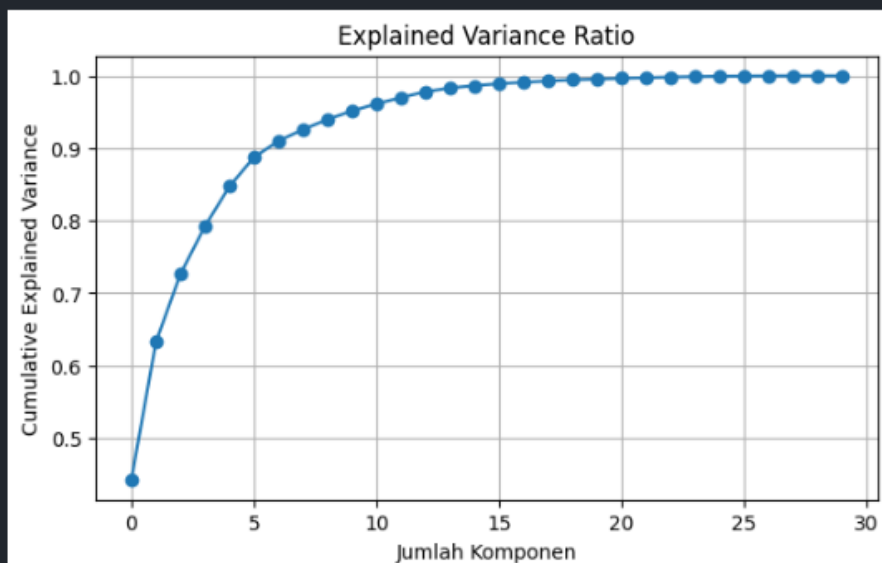
6. Grafik Explained Variance

```
X_scaled = StandardScaler().fit_transform(X)

pca_full = PCA()
pca_full.fit(X_scaled)

explained_variance = pca_full.explained_variance_ratio_

plt.figure(figsize=(7,4))
plt.plot(np.cumsum(explained_variance), marker='o')
plt.xlabel("Jumlah Komponen")
plt.ylabel("Cumulative Explained Variance")
plt.title("Explained Variance Ratio")
plt.grid(True)
plt.show()
```



Analisis Grafik

Berdasarkan grafik *Cumulative Explained Variance*, terlihat bahwa kurva mengalami kenaikan yang sangat tajam pada komponen-komponen awal. Hal ini menunjukkan bahwa sebagian besar informasi dalam dataset Breast Cancer sudah dapat direpresentasikan oleh beberapa komponen utama pertama hasil PCA.

Dari grafik dapat diamati bahwa:

- Sekitar **5 komponen** sudah mampu menjelaskan kurang lebih **89% variasi data**.
- Sekitar **7 komponen** sudah mampu menjelaskan sekitar **90% variasi data**.
- Sekitar **10 komponen** mampu menjelaskan sekitar **95% variasi data**.
- Setelah lebih dari **15 komponen**, kenaikan explained variance menjadi sangat kecil karena sebagian besar informasi penting sudah berhasil ditangkap oleh komponen-komponen sebelumnya.

Hasil ini menunjukkan bahwa PCA mampu mereduksi dimensi data dari **30 fitur menjadi sekitar 7–10 komponen** tanpa kehilangan sebagian besar informasi yang terkandung dalam dataset. Oleh karena itu, penggunaan PCA dapat membantu menyederhanakan data, mengurangi kompleksitas model, serta mempercepat proses komputasi tanpa menurunkan performa secara signifikan.

7. Kesimpulan Komparatif

Berdasarkan seluruh percobaan, PCA memberikan pengaruh yang berbeda pada setiap model machine learning. Model berbasis jarak seperti KNN cenderung memperoleh manfaat yang lebih besar karena PCA membantu mengurangi dampak *curse of dimensionality*. Logistic Regression umumnya mengalami perubahan performa yang lebih kecil karena model ini sudah cukup efektif dalam menangani data berdimensi tinggi.

Penggunaan PCA dengan jumlah komponen yang terlalu sedikit, seperti 2 komponen, dapat menyebabkan hilangnya informasi penting sehingga akurasi model menurun. Sebaliknya, penggunaan PCA dengan jumlah komponen yang lebih banyak, seperti 10 komponen atau jumlah komponen yang mempertahankan 90% variasi data, biasanya mampu menjaga performa model tetap baik sambil mengurangi kompleksitas data.

PCA terbukti efektif sebagai teknik reduksi dimensi, tetapi jumlah komponen yang digunakan harus dipilih secara tepat agar diperoleh keseimbangan antara efisiensi komputasi dan performa klasifikasi.

PERTANYAAN REFLEKTIF

1. Mengapa PCA termasuk unsupervised learning?

PCA termasuk metode *unsupervised learning* karena proses pembentukan komponen utama tidak menggunakan informasi label atau target kelas. PCA hanya memanfaatkan hubungan dan variasi antar fitur untuk menemukan arah variasi terbesar dalam data. Oleh karena itu, PCA dapat diterapkan tanpa mengetahui kategori atau kelas dari setiap data.

2. Mengapa PCA membantu model berbasis jarak?

PCA membantu model berbasis jarak seperti KNN karena mampu mengurangi jumlah dimensi dan menghilangkan fitur yang redundan. Pada data berdimensi tinggi, jarak antar data menjadi kurang representatif akibat fenomena *curse of dimensionality*. Dengan mereduksi dimensi, PCA membuat perhitungan jarak menjadi lebih efektif sehingga proses klasifikasi dapat dilakukan dengan lebih baik.

3. Kapan PCA justru menurunkan performa?

PCA dapat menurunkan performa ketika jumlah komponen yang dipilih terlalu sedikit sehingga sebagian informasi penting ikut terbuang. Selain itu, PCA hanya mempertimbangkan variasi data tanpa memperhatikan target kelas, sehingga ada kemungkinan fitur yang penting untuk klasifikasi justru tidak dipertahankan. Akibatnya, akurasi model dapat menurun setelah proses reduksi dimensi.

4. Apakah PCA selalu diperlukan sebelum ensemble?

Tidak. PCA tidak selalu diperlukan sebelum model ensemble seperti Random Forest atau Gradient Boosting. Model ensemble umumnya mampu menangani data berdimensi tinggi dengan baik dan memiliki mekanisme internal untuk memilih fitur yang penting. Penggunaan PCA biasanya lebih bermanfaat pada model yang sensitif terhadap jumlah dimensi atau berbasis jarak, sedangkan pada beberapa model ensemble PCA justru dapat menghilangkan informasi yang berguna.

5. Bagaimana PCA berkaitan dengan curse of dimensionality?

Curse of dimensionality adalah kondisi ketika jumlah fitur yang sangat banyak menyebabkan data menjadi semakin kompleks dan sulit diproses oleh model machine learning. PCA membantu mengatasi masalah ini dengan mengurangi jumlah dimensi data menjadi beberapa komponen utama yang tetap mempertahankan sebagian besar informasi penting. Dengan jumlah dimensi yang lebih sedikit, model menjadi lebih efisien, proses komputasi lebih cepat, dan risiko overfitting dapat berkurang.